

HOBBY TRACKER

Build Good Routines - One Streak at a Time

Course: Object-Oriented and Functional Programming with Python

DLBDSOOFPP01

IU International University of Applied Sciences

Student Name: Dikshya Khatri

Matriculation Number: 10235413

Date: 11.05.2026

Application Overview

What is Hobby Tracker?

A Python CLI application that lets users define personal hobbies with a recurrence cycle and automatically tracks how consistently they complete each one, visualised as a live streak counter and ASCII progress bar.

Core Features

- Create, edit, and remove habits
- Log a completed session with one key press
- View ASCII progress bars (current vs personal best)
- Analyse habits with functional-style aggregations
- All data persisted in a JSON file between sessions

Technology Stack

- **Language:** Python 3.7+
- **Paradigms:** OOP + Functional Programming
- **Persistence:** JSON file (built-in json module)
- **CLI:** Built-in input() - no extra deps
- **Analytics:** map / filter / reduce / sorted / max
- **Testing:** unittest (standard library)
- **Hosting:** GitHub public repository

Architecture Design

Component Summary

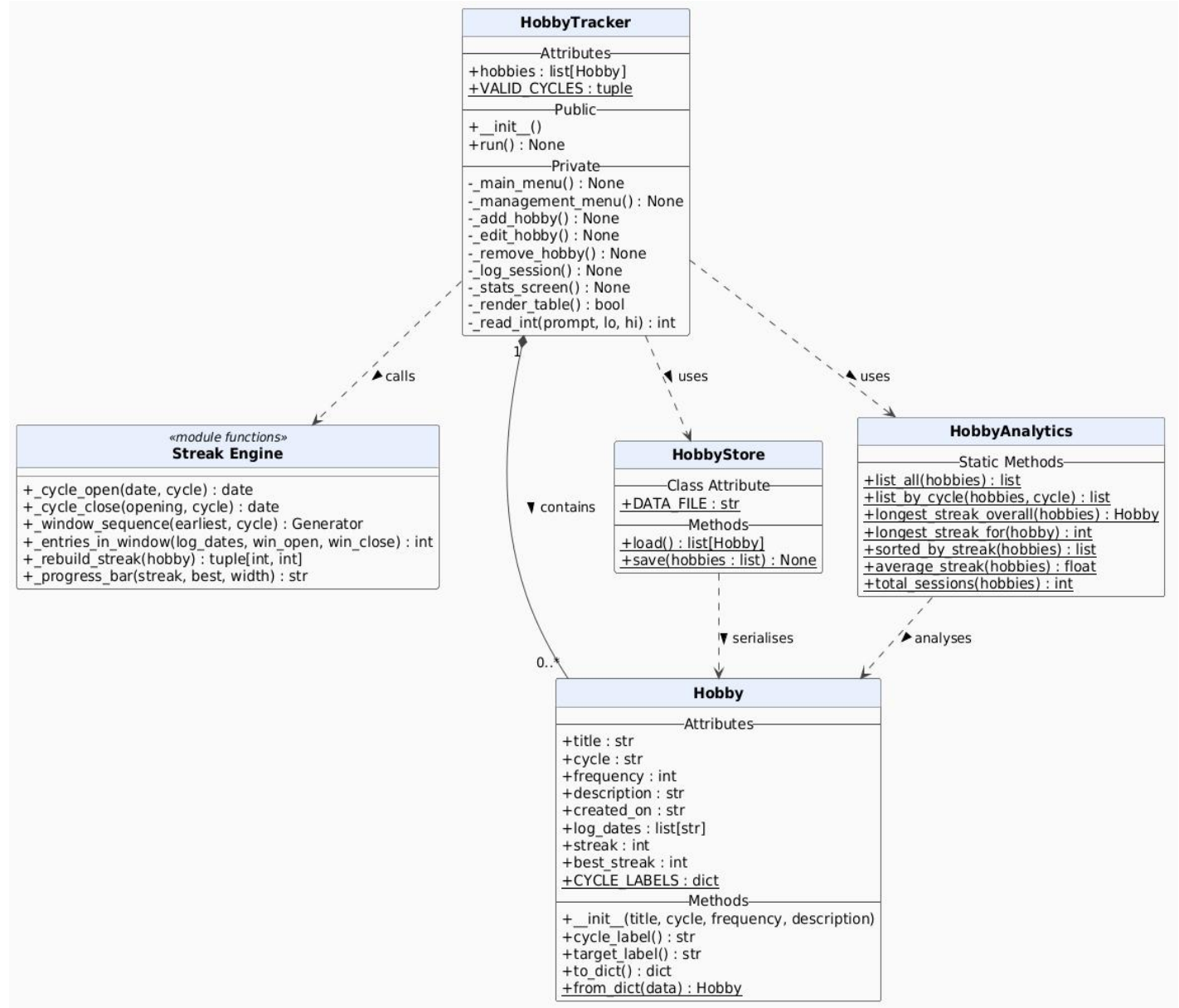
Hobby: Data model - one personal habit record with cycle, frequency, streaks and log dates.

HobbyStore: Persistence - JSON load/save; singleton class attribute DATA_FILE.

HobbyAnalytics: Functional analytics – map, filter, max, reduce, sorted.

HobbyTracker: CLI controller – 4-option main menu, sub-menus, table renderer

Streak Engine: Pure module-level functions - cycle windows, streak rebuild, ASCII bar.



The Hobby Class - Data Model

Attributes

- **Title(str)** - Unique hobby name
- **cycle(str)** - daily / weekly / monthly / yearly
- **Frequency** – Min. sessions per cycle to keep streak
- **Description** – Free-text note about the hobby
- **created_on** – Auto-stamped YYYY-MM-DD HH:MM
- **Log_dates** - ISO dates of every logged session
- **Streak** - Consecutive completed cycles
- **Best_streak** – All-time personal best streak
- **CYCLE_LABELS** - Class constant - display labels

Key Methods

```
class Hobby:
    CYCLE_LABELS = {
        'daily': 'Daily',
        'weekly': 'Weekly',
        'monthly': 'Monthly',
        'yearly': 'Yearly',
    }

    def __init__(self, title, cycle,
                  frequency, description):
        self.title = title
        self.cycle = cycle
        self.frequency = frequency
        self.log_dates = []
        self.streak = 0
        self.best_streak = 0

    def to_dict(self): ...
    @classmethod
    def from_dict(cls, data): ...
```

Persistence & Analytics Layers

Hobby Store - Persistence

Manages all file-system I/O. Hobbies are written to hobbies.json under a top-level “hobbies” key so future metadata can be added without breaking the existing format.

```
class HobbyStore:
```

```
    DATA_FILE = 'hobbies.json'
```

```
    @classmethod
```

```
    def load(cls) -> list[Hobby]:
```

```
        if not os.path.exists(cls.DATA_FILE):
```

```
            return []
```

```
        with open(cls.DATA_FILE) as fh:
```

```
            payload = json.load(fh)
```

```
        return [Hobby.from_dict(r)
```

```
                for r in payload['hobbies']]
```

```
    @classmethod
```

```
    def save(cls, hobbies):
```

```
        payload = {'hobbies':
```

```
                    [h.to_dict() for h in hobbies]}
```

```
        with open(cls.DATA_FILE, 'w') as fh:
```

```
            json.dump(payload, fh, indent=4)
```

HobbyAnalytics - Functional

Stateless class using higher-order functions exclusively, no imperative loops.

- **List_all(hobbies)**
- **Longest_streak_overall(h)**
- **List_by_cycle(hobbies, c)**
- **Longest_streak_for(hobby)**
- **Sorted_by_streak(hobbies)**
- **Average_streak(hobbies)**
- **Total_sessions(hobbies)**

Streak Engine

How streaks are calculated

- `_cycle_open(date, cycle)` - first day of the window
- `_cycle_close(opening, cycle)` - last day of the window
- `_window_sequence(earliest, cycle)` - generator of all (open, close) pairs
- `_entries_in_window(log_dates, open, close)` - sessions inside this window
- `_rebuild_streak(hobby)` - walks every window, counts hits/misses, returns (streak, best)
- `_progress_bar(streak, best, width)` - ASCII  bar string

Key insight: a still-open window ($win_close \geq today$) that is below target does NOT break the streak - the user still has time to log.

```
def _rebuild_streak(hobby):
    if not hobby.log_dates:
        return 0, hobby.best_streak

    earliest = datetime.strptime(
        sorted(hobby.log_dates)[0], '%Y-%m-%d').date()
    today = datetime.now().date()
    current, best = 0, hobby.best_streak

    for win_open, win_close in \
        _window_sequence(earliest, hobby.cycle):
        count = _entries_in_window(
            hobby.log_dates, win_open, win_close)

        if count >= hobby.frequency:
            current += 1
            if current > best:
                best = current

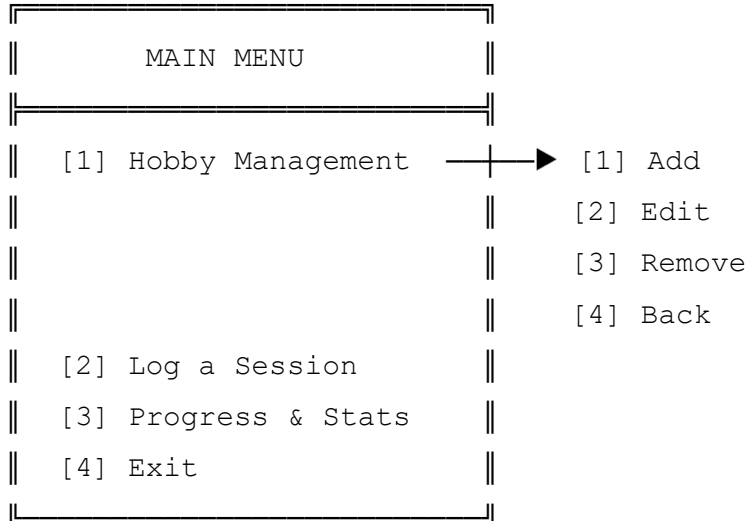
        elif win_close < today: # past, missed
            current = 0

        # open window → no reset

    return current, best
```

Command-Line Interface & User Experience

Menu Structure



UX Safety Nets

- Confirmation preview box before any new hobby is saved
- Duplicate-name guard on add
- Per-day cap on daily hobbies (max 1 session / day)
- Frequency cap on non-daily hobbies
- `_read_int()` centralises all numeric input validation

Progress & Stats Screen

Progress (current streak vs personal best)

Morning Journaling  12/20

Chess Puzzles  4/ 4

Apartment Clean  3/ 4

Total hobbies : 3

Total sessions : 47

Avg streak : 6.3

Top streak : Morning Journaling — best 20

Hobbies by cycle

Daily : Morning Journaling

Weekly : Chess Puzzles

Monthly : Apartment Clean

Streak leaderboard

#1 Morning Journaling best: 20 sessions: 35

#2 Apartment Clean best: 4 sessions: 8

#3 Chess Puzzles best: 4 sessions: 4

Unit Test Suite

Test Classes (unittest)

TestHobby - 15 tests (Construction, serialization, round-trip)

TestHobbyAnalytics - 13 tests (map / filter / max / reduce / sorted)

TestHobbyStore - 5 tests (Save - reload round-trips; missing file)

TestHobbyTracker - 11 tests (Streak engine, progress bar, controller)

Total: 44 tests

Key Takeaways

- **OOP:** Hobby, HobbyStore, HobbyAnalytics and HobbyTracker each have a single, well-defined responsibility.
- **Functional:** All analytics use map, filter, reduce, max and sorted - zero imperative loops in the analytics layer.
- **Streak Engine:** Pure module-level functions make the streak algorithm independently testable and easy to reason about.
- **Persistence:** JSON round-trip via HobbyStore keeps the app dependency-free while supporting all four cycle types.
- **Testing:** 44 unit tests cover construction, serialization, persistence, streak edge-cases and analytics.